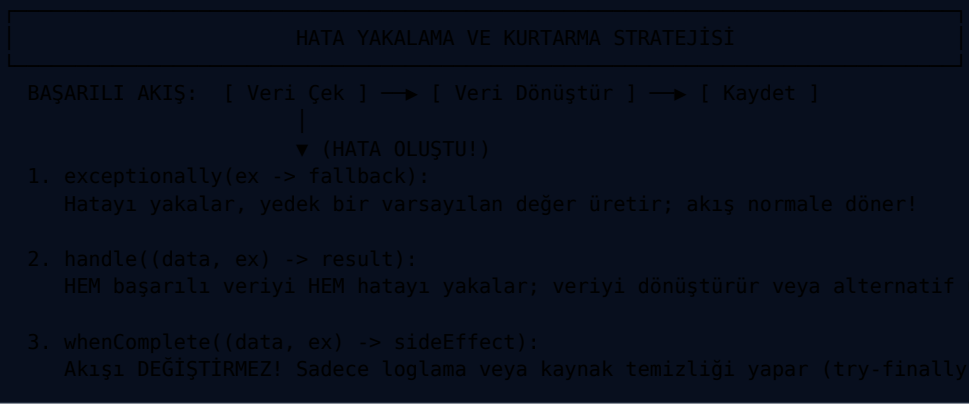
41

BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Hata Kurtarma Zincirleri: exceptionally vs handle vs whenComplete ile Dayanıklılık

Resilience & Exception Handling: exceptionally vs handle vs whenComplete

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kargo teslimatı düşünün: 1) exceptionally: Müşteri evde yoksa kargoyu kapıcıya bırakmaktır (hata olursa yedek plan devreye girer, paket teslim edilmiş sayılır). 2) handle: Kuryenin kapıyı çaldıktan sonra 'Paketi teslim ettim' veya 'Evde yoktu, şubeye yönlendirdim' diye merkez sisteme rapor hazırlamasıdır (hem başarıyı hem hatayı tek bir yeni sonuca çevirir). 3) whenComplete: Paket teslim edilse de kaybolsa da kuryenin araba günlüğüne 'Bu adrese uğrandı' diye imza atmasıdır (akışı değiştirmez, sadece log tutar).

🇬🇧 ENGLISH (ELI5)

1) `exceptionally`: Like a fallback generator. If the primary power grid cuts out, the backup diesel generator fires up seamlessly, supplying replacement power downstream. 2) `handle`: An inspection checkpoint. It receives both `(result, throwable)`. Whether the product is pristine or shattered, it packs it into an inspected report object for downstream consumers. 3) `whenComplete`: Pure side-effect auditing (like `finally`). It logs whether the transaction succeeded or threw an error, but re-emits the exact upstream state downstream without alteration.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Metot İmzaları ve İstisna Kapsülleme (CompletionException): CompletableFuture içindeki kontrol edilmeyen hatalar HotSpot tarafından otomatik olarak `CompletionException` ile sarılır. 1) `exceptionally(Function\<Throwable, T>)`: Yalnızca istisna durumunda tetiklenir. İstisnayı yutar ve geriye geçerli bir `T` değeri döndürerek pipeline'ı kurtarır. 2) `handle(BiFunction\<T, Throwable, U>)`: İstisna olsun veya olmasın her durumda tetiklenir. Eğer `ex != null` ise hata işlenir, aksi halde `res` işlenir. Yeni bir `U` tipi döndürür. 3) `whenComplete(BiConsumer\<T, Throwable>)`: Gözlemcidir (side-effect). Geriye değer döndürmez (`void`). Orijinal stage'in sonucunu veya istisnasını aynen sonraki aşamalara geçirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Exception Handling Mechanics: Unchecked exceptions in asynchronous pipelines are wrapped in `CompletionException`. 1) `exceptionally`: Invoked exclusively on stage failure. Catches the `Throwable`, recovers with an alternative value `T`, and transitions the downstream stage to normal completion. 2) `handle`: Invoked unconditionally on dual outcomes (`T result, Throwable ex`). Transforms either state into `U`. 3) `whenComplete`: Passive terminal/audit stage taking `BiConsumer\<T, Throwable>`. It executes side-effects (metrics, auditing) without intercepting or altering the original value/exception payload.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: CompletableFuture Dayanıklı Hata Kurtarma
import java.util.concurrent.CompletableFuture;

public class ResilientPipelineDemo {
    record Price(String symbol, double amount, boolean isFallback) {}

    static CompletableFuture<Double> fetchLivePriceFromExchange(String symbol) {
        return CompletableFuture.supplyAsync(() -> {
            if ("BTC".equals(symbol)) {
                throw new IllegalStateException("Kripto Borsası Çöktü (503 Service Unavailable)!");
            }
            return 65000.0;
        });
    }

    void main() {
        String symbol = "BTC";

        CompletableFuture<Price> priceFuture = fetchLivePriceFromExchange(symbol)
            // 1. exceptionally ile hatayı yakala ve önbellekten fallback dön!
            .exceptionally(ex -> {
                System.err.println("⚠ Hata Yakalandı: " + ex.getMessage() + ". Önbelleğe
dönülüyor...");
                return 62000.0; // Cached fallback price
            })
            // 2. handle ile veriyi modelle
            .handle((val, ex) -> {
                boolean isFallback = (val == 62000.0);
                return new Price(symbol, val, isFallback);
            })
            // 3. whenComplete ile metrik logla (akışı bozmaz!)
            .whenComplete((price, ex) -> {
                System.out.println("📊 Audit Log: Fiyat başarıyla belirlendi -> " + price);
            });

        Price finalPrice = priceFuture.join();
        System.out.println("Sonuç: " + finalPrice);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

exceptionallyAsync kullanmazsanız, hata anında fallback üretme kodu hatanın patladığı worker thread üzerinde çalışır. Fallback kodu veritabanına veya diske gidiyorsa thread'i kilitler; bu tür I/O kurtarmalarında exceptionallyCompose veya exceptionallyAsync tercih edilmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Unless `exceptionallyAsync` is used, the fallback logic executes synchronously on whichever worker thread encountered the upstream exception. If your fallback invokes blocking I/O (e.g. disk or cache reads), use `exceptionallyComposeAsync` to avoid carrier thread exhaustion.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• exceptionally hatayı varsayılan değerle kurtarır. handle hem hatayı hem başarıyı dönüştürür. whenComplete ise yan etkisiz loglama yapar.

🎯 Key Takeaways for the Interview

• `exceptionally` recovers from failure with a fallback. `handle` translates dual states into a new value. `whenComplete` performs side-effects without altering state.